

1. Find all anagrams of string p in s.

Question: Given two strings, s and p, find all the starting indices in s where an anagram of p exists as a substring.

Simple Explanation: We use a **Sliding Window** of the same length as string p. We maintain a **frequency map** (or array) of characters for both p and the current window in s. As the window slides one step to the right, we subtract the outgoing character's frequency and add the incoming character's frequency. If the two frequency maps match, we record the window's starting index.

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class AnagramFinder {
    public static List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();
        if (s == null || p == null || s.length() < p.length()) {
            return result;
        }

        int n = s.length();
        int k = p.length();

        // Use arrays of size 26 for simple character counting (A-Z or a-
z)
        int[] pCount = new int[26];
        int[] sCount = new int[26];

        // 1. Initialize the target count (p) and the first window's count
(s[0..k-1])
        for (int i = 0; i < k; i++) {
            pCount[p.charAt(i) - 'a']++;
            sCount[s.charAt(i) - 'a']++;
        }

        // Check the first window
        if (Arrays.equals(pCount, sCount)) {
            result.add(0);
        }

        // 2. Slide the window from index k to n-1
        for (int i = k; i < n; i++) {
            // Add the new incoming character (right side)
            sCount[s.charAt(i) - 'a']++;
            // Remove the old outgoing character (left side)
            sCount[s.charAt(i - k) - 'a']--;

            // Check if the counts match after the slide
            if (Arrays.equals(pCount, sCount)) {
                result.add(i - k + 1); // Record the start index
            }
        }
    }
}
```

```

        }
    }
    return result;
}
}

```

2. Minimum window substring.

Question: Given two strings, s (the source) and t (the target), find the smallest substring in s that contains all characters of t, including duplicates.

Simple Explanation: We use a **Sliding Window** combined with a **frequency map** for the target string t. We expand the window (right pointer) until all required characters of t are covered. Once covered, we try to shrink the window (left pointer) as much as possible while maintaining the coverage. We update the minimum length found at each valid window contraction.

```

import java.util.HashMap;
import java.util.Map;

public class MinWindowSubstring {
    public static String minWindow(String s, String t) {
        if (s == null || t == null || s.length() < t.length()) {
            return "";
        }

        // Map to store character frequencies of the target string t
        Map<Character, Integer> targetCounts = new HashMap<>();
        for (char c : t.toCharArray()) {
            targetCounts.put(c, targetCounts.getOrDefault(c, 0) + 1);
        }

        int required = targetCounts.size(); // Number of unique characters
        int formed = 0; // Number of unique characters
        int left = 0; // currently covered
        int minLength = Integer.MAX_VALUE;
        int minStart = 0;

        // Map to store character frequencies of the current window in s
        Map<Character, Integer> windowCounts = new HashMap<>();

        // Expand the window with the right pointer
        for (int right = 0; right < s.length(); right++) {
            char charR = s.charAt(right);

            // 1. Update window counts and check if a requirement is met
            windowCounts.put(charR, windowCounts.getOrDefault(charR, 0) +
1);

```

```

        if (targetCounts.containsKey(charR)
windowCounts.get(charR).intValue()
targetCounts.get(charR).intValue()) {
            formed++;
        }

        // 2. Try to contract the window from the left once all
characters are covered (formed == required)
        while (left <= right && formed == required) {
            // Update minimum length found
            int currentLength = right - left + 1;
            if (currentLength < minLength) {
                minLength = currentLength;
                minStart = left;
            }

            // Remove the character sliding out (charL)
            char charL = s.charAt(left);
            windowCounts.put(charL, windowCounts.get(charL) - 1);

            // Check if removing the character breaks the requirement
            if (targetCounts.containsKey(charL)
windowCounts.get(charL).intValue() < targetCounts.get(charL).intValue())
            {
                formed--;
            }

            left++; // Shrink the window
        }
    }

    return minLength == Integer.MAX_VALUE ? "" : s.substring(minStart,
minStart + minLength);
}
}

```

3. Number of substrings with at most k distinct characters.

Question: Given a string *s* and an integer *k*, find the total number of substrings (continuous blocks) that contain no more than *k* unique characters.

Simple Explanation: We use the **Sliding Window** technique combined with a **HashMap** to count the distinct characters in the current window. We expand the window (right pointer) as much as possible while the number of distinct characters is *k*. Each time the window expands successfully, every substring ending at the right pointer and starting anywhere between *left* and *right* is a valid substring, adding (*right* - *left* + 1) to the total count.

```

import java.util.HashMap;
import java.util.Map;

public class SubstringsWithKDistinct {

```

```

public static int countSubstrings(String s, int k) {
    if (s == null || s.isEmpty() || k == 0) {
        return 0;
    }

    Map<Character, Integer> charCounts = new HashMap<>();
    int left = 0;
    int totalCount = 0;

    for (int right = 0; right < s.length(); right++) {
        char charR = s.charAt(right);

        // Add the new character to the window
        charCounts.put(charR, charCounts.getOrDefault(charR, 0) + 1);

        // While the number of distinct characters is greater than k,
        shrink the window
        while (charCounts.size() > k) {
            char charL = s.charAt(left);

            // Remove the character sliding out
            charCounts.put(charL, charCounts.get(charL) - 1);

            // If the count hits zero, remove the character from the
            distinct set
            if (charCounts.get(charL) == 0) {
                charCounts.remove(charL);
            }

            left++; // Shrink the window from the left
        }

        // At this point, the window [left...right] is valid.
        // All substrings ending at 'right' and starting at or after
        'left' are valid.
        totalCount += (right - left + 1);
    }
    return totalCount;
}

```

4. Count subarrays divisible by k.

Question: Given an array of integers nums, return the number of non-empty subarrays (continuous blocks) that have a sum divisible by k.

Simple Explanation: We do not use the standard sliding window. We use a mathematical trick based on the **prefix sum** and the **modulo operator**. If the prefix sum at index i and the prefix sum at index j have the same remainder when divided by k, then the subarray between i and j must be divisible by k. We use a **HashMap** to count how many times each remainder occurs.

$(\text{prefix_sum}[j] - \text{prefix_sum}[i-1]) \bmod k = 0 \implies \text{prefix_sum}[j] \bmod k = \text{prefix_sum}[i-1] \bmod k$

```

import java.util.HashMap;
import java.util.Map;

public class SubarraysDivisibleByK {
    public static int subarraysDivByK(int[] nums, int k) {
        if (nums == null || k == 0) {
            return 0;
        }

        // Map to store (remainder -> count of times this remainder has
        // been seen)
        Map<Integer, Integer> remainderCounts = new HashMap<>();
        remainderCounts.put(0, 1); // Initialize count for remainder 0
        // (for the case where prefix_sum[j] is divisible by k)

        int currentPrefixSum = 0;
        int totalSubarrays = 0;

        for (int num : nums) {
            currentPrefixSum += num;

            // Calculate the remainder. The outer k+ is used to handle
            // negative numbers in Java.
            int remainder = (currentPrefixSum % k + k) % k;

            // If this remainder has been seen before, it means the
            // difference (the subarray sum) is divisible by k.
            if (remainderCounts.containsKey(remainder)) {
                // Add the count of times this remainder has occurred
                // previously
                totalSubarrays += remainderCounts.get(remainder);
            }

            // Update the count for the current remainder
            remainderCounts.put(remainder,
                remainderCounts.getOrDefault(remainder, 0) + 1);
        }

        return totalSubarrays;
    }
}

```

5. Longest repeating character replacement.

Question: Given a string *s* and an integer *k* (the maximum number of times you can change any character), find the length of the longest substring you can form by changing at most *k* characters to a single, repeating character.

Simple Explanation: We use a **Sliding Window** where we want the window to contain mostly one character, allowing up to k other characters to be replaced. We track the **highest frequency character count** (max_freq) within the current window. The window is valid if the total number of characters that are *not* the most frequent character is k . We only shrink the window (left pointer) when the window becomes invalid.

Window is valid if: $(\text{Window Length}) - (\text{Max Character Frequency}) \leq k$

```
import java.util.HashMap;
import java.util.Map;

public class LongestRepeatingReplacement {
    public static int characterReplacement(String s, int k) {
        if (s == null || s.isEmpty()) {
            return 0;
        }

        Map<Character, Integer> charCounts = new HashMap<>();
        int left = 0;
        int maxLength = 0;
        int maxFreq = 0; // Highest frequency of any character in the
        // current window

        for (int right = 0; right < s.length(); right++) {
            char charR = s.charAt(right);

            // 1. Expand the window and update frequency
            charCounts.put(charR, charCounts.getOrDefault(charR, 0) + 1);

            // Track the maximum frequency found so far in the current
            // window
            maxFreq = Math.max(maxFreq, charCounts.get(charR));

            // 2. Check if the window is invalid
            // Invalid if: (Window Size - Max Frequency) > k
            // (meaning we have more than k characters that need
            // replacement)
            int windowLength = right - left + 1;

            if (windowLength - maxFreq > k) {
                // Shrink the window: remove the character sliding out
                char charL = s.charAt(left);
                charCounts.put(charL, charCounts.get(charL) - 1);

                left++; // Advance the left pointer

                // Note: We don't need to recalculate maxFreq accurately
                // every time we shrink;
                // only checking window length against the current maxFreq
                // is enough to ensure validity.
            }
        }
    }
}
```

```

        // 3. Update the global maximum length found
        windowLength = right - left + 1; // Recalculate length after
potential shrink
        maxLength = Math.max(maxLength, windowLength);
    }

    return maxLength;
}
}

```

6. Sliding window maximum.

Question: Given an array `nums` and a sliding window size `k`, find the maximum value in the window as it slides one position to the right.

Simple Explanation: We use a specialized data structure called a **Deque (Double-Ended Queue)** to store the indices of elements in the current window. The deque maintains the indices of potentially maximum elements in descending order. This allows us to find the maximum value of the current window in $O(1)$ time by just looking at the front of the deque.

```

import java.util.ArrayDeque;
import java.util.Deque;

public class SlidingWindowMaximum {
    public static int[] maxSlidingWindow(int[] nums, int k) {
        if (nums == null || k <= 0 || nums.length < k) {
            return new int[0];
        }

        int n = nums.length;
        int[] results = new int[n - k + 1];
        // Deque stores indices of elements; the front is always the index
of the largest element in the current window.
        Deque<Integer> deque = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            // 1. Remove indices of elements that are outside the current
window [i-k+1, i]
            if (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
                deque.pollFirst();
            }

            // 2. Remove indices of smaller elements from the back
            // (If the new element is larger, the older, smaller elements
are no longer useful)
            while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[i])
            {
                deque.pollLast();
            }

```

```

        // 3. Add the current element's index to the back
        deque.offerLast(i);

        // 4. Record the result after the first full window is formed
        if (i >= k - 1) {
            // The maximum element's index is always at the front of
the deque
            results[i - k + 1] = nums[deque.peekFirst()];
        }
    }
    return results;
}
}

```

7. Count distinct elements in every window of size k.

Question: Given an array `arr` and a window size `k`, count the number of distinct (unique) elements in each sliding window of size `k`.

Simple Explanation: We use a **Sliding Window** with a **HashMap** to store the frequency of elements in the current window. When the window slides, we subtract the frequency of the outgoing element and add the frequency of the incoming element. We check the size of the HashMap at each step to get the count of distinct elements.

```

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class DistinctElementsInWindow {
    public static List<Integer> countDistinct(int[] arr, int k) {
        List<Integer> result = new ArrayList<>();
        if (arr == null || k > arr.length) {
            return result;
        }

        // Map stores (element -> frequency) for the current window
        Map<Integer, Integer> windowFreq = new HashMap<>();

        // 1. Process the first window (initial setup)
        for (int i = 0; i < k; i++) {
            windowFreq.put(arr[i], windowFreq.getOrDefault(arr[i], 0) +
1);
        }
        result.add(windowFreq.size()); // Size of the map is the distinct
count

        // 2. Slide the window across the rest of the array
        for (int i = k; i < arr.length; i++) {
            // Remove the outgoing element (left side: arr[i - k])
            int outgoing = arr[i - k];

```



```

        windowFreq.put(outgoing, windowFreq.get(outgoing) - 1);

        // If the count of the outgoing element hits zero, remove it
from the map
        if (windowFreq.get(outgoing) == 0) {
            windowFreq.remove(outgoing);
        }

        // Add the incoming element (right side: arr[i])
        int incoming = arr[i];
        windowFreq.put(incoming, windowFreq.getOrDefault(incoming,
0) + 1);

        // The map size (windowFreq.size()) gives the new distinct
count
        result.add(windowFreq.size());
    }

    return result;
}
}

```

8. Smallest subarray with sum greater than a given value.

Question: Given an array of positive integers `nums` and a target sum `target`, find the length of the smallest continuous subarray whose sum is greater than or equal to `target`.

Simple Explanation: We use the **Sliding Window** technique with a flexible window size. We expand the window (right pointer) and accumulate the sum. Once the sum meets or exceeds the target, we record the current window size and then immediately try to shrink the window (left pointer) to find the minimum possible length while the sum remains valid.

```

public class MinSubarraySum {
    public static int minSubArrayLen(int target, int[] nums) {
        if (nums == null || nums.length == 0) {
            return 0;
        }

        int left = 0;
        int currentSum = 0;
        int minLength = Integer.MAX_VALUE;

        // Right pointer expands the window
        for (int right = 0; right < nums.length; right++) {
            currentSum += nums[right];

            // While the sum is valid, try to shrink the window from the
left
            while (currentSum >= target) {
                // Record the current minimum length
                int currentLength = right - left + 1;
            }
        }
    }
}

```

```
        minLength = Math.min(minLength, currentLength);

        // Shrink the window by removing the left element
        currentSum -= nums[left];
        left++;
    }

    // If minLength is still MAX_VALUE, no such subarray was found
    return minLength == Integer.MAX_VALUE ? 0 : minLength;
}
```

JAVA Interview Buddy